

SISTEMAS OPERATIVOS 2

Jeyson gonzalez vinasco

GRUPO 102

Simulador de CPU

contexto: Imagina que el sistema SIGET controla el tráfico de toda la ciudad es el único cajero en un banco que siempre está lleno. Las "tareas" o procesos son los clientes. Si el cajero atiende mal o se enreda con un cliente demorado, el banco colapsa (es decir, el tráfico se detiene). Esta simulación demuestra cómo enseñarle a nuestro cajero (la CPU) a organizar la fila de forma inteligente para que la movilidad de la ciudad fluya sin problemas.

1. Objetivos y Decisiones de Diseño

- **Objetivo Principal:** Diseñar e implementar un simulador en Python que demuestre cómo el "cerebro" (CPU) del SIGET gestiona diferentes tareas para evitar sobrecargas y garantizar que no haya colapso en la ciudad.
- **Decisiones de Diseño:**
 - **Tipos de Tareas:** Para hacerlo realista, creamos tres tipos de procesos: de *rutina* (ej. leer un semáforo normal), de *emergencia* (ej. choque de carros) y *pesados* (ej. analizar muchos datos a la vez).
 - **Ciclo de vida (Los 5 estados):** Se programó para que cada tarea pase por 5 etapas como en la vida real: *Nuevo* (llega), *Listo* (hace fila), *En ejecución* (lo atienden), *Bloqueado* (pausado esperando otra cosa) y *Terminado*.

2. Algoritmos Utilizados y Observaciones Para organizar la fila, usamos dos métodos distintos:

- **Algoritmo de Prioridad (El "Pase VIP"):** Le da una etiqueta de urgencia a cada tarea. Las de alta urgencia (emergencias) se atienden primero.
 - *Observación:* Es perfecto para las alertas críticas porque responde de inmediato. Sin embargo, si hay demasiadas emergencias, las tareas normales nunca son atendidas (se quedan esperando por siempre).
- **Algoritmo Round-Robin (La "Ronda de Turnos Rápidos"):** Le da a todos un tiempo máximo estricto (un "Quantum"). Si el tiempo se acaba y no has terminado, vuelves al final de la fila.

- *Observación:* Es muy equitativo. Evitó que el procesamiento de datos pesados acaparara toda la atención, permitiendo que la rutina siguiera avanzando.

3. Conclusión: Desempeño en el contexto del SIGET Para que el SIGET funcione y la movilidad urbana no colapse, **ningún algoritmo por sí solo es suficiente**. Si usáramos solo *Prioridad*, las ambulancias avanzarían perfecto, pero el procesamiento normal de los semáforos se congelaría y habría embotellamientos. Si usáramos solo *Round-Robin*, todo avanzaría equitativamente, pero trataríamos una emergencia crítica con la misma importancia que la lectura de un sensor de rutina.

Conclusión: El SIGET necesita un **sistema híbrido**. Debe aplicar *Prioridad* absoluta para dar respuesta inmediata a accidentes y bloqueos, y al mismo tiempo utilizar *Round-Robin* para organizar la inmensa cantidad de datos de tráfico diarios, equilibrando así rapidez y eficiencia general.